sensors

MDPI

Article

A Lightweight ECC-Based Authentication and Key Agreement Protocol for IoT with Dynamic Authentication Credentials

Momo Li and Shunfang Hu *

School of Mathematics and Computer Science, Yunnan Minzu University, Kunming 650504, China; 040276@ymu.edu.cn

* Correspondence: hsf@ymu.edu.cn

Abstract: Due to the openness of communication channels and the sensitivity of the data being collected and transmitted, securing data access and communication in IoT systems requires robust ECC-based authentication and key agreement (AKA) protocols. However, designing an AKA protocol for IoT presents significant challenges, as most IoT sensors are deployed in resource-constrained, unattended environments with limited computational power, connectivity, and storage. To achieve anonymous authentication, existing solutions typically rely on shared temporary public keys to mask device IDs or validate sender certificates, which increases the computational overhead. Furthermore, these protocols often fail to address crucial security concerns, such as nonresistance to ephemeral secret leakage (ESL) attacks and a lack of perfect forward security. To mitigate the computational burden, we propose a dynamic authenticated credentials (DACs) synchronization framework for anonymous authentication. Then, we introduce an ECC-based AKA scheme that employs DACs in place of temporary public keys or sender credentials, enabling efficient and secure anonymous authentication. The security of the proposed protocol was rigorously verified under the Real-or-Oracle model and validated using ProVerif. Performance comparisons demonstrate that our scheme offered significant improvements in security, with an over 37% reduction in communication cost and computational overhead.

Keywords: internet of things; anonymity; authentication and key agreement; dynamic authenticated credentials; ephemeral information leakage attacks

check for updates

Citation: Li, M.; Hu, S. A Lightweight ECC-Based Authentication and Key Agreement Protocol for IoT with Dynamic Authentication Credentials. *Sensors* **2024**, *24*, 7967. https://doi.org/10.3390/s24247967

Academic Editors: Rolando Herrero and Abhishek Murthy

Received: 30 October 2024
Revised: 5 December 2024
Accepted: 12 December 2024
Published: 13 December 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The Internet of Things (IoT) has revolutionized the way devices, machines, and individuals connect and interact, enabling seamless interoperability regardless of time or location. IoT applications are typically categorized into time-driven and event-driven applications [1]. In event-driven applications, sensors detect specific events, such as the movement or appearance of the target, within a predefined area. Upon detecting such events, the data are transmitted through specialized network architectures to ensure timely communication. In contrast, time-driven IoT applications, as illustrated in Figure 1, involve sensors that periodically collect data, such as temperature, pressure, and humidity, and transmit them to central servers. This process, referred to as periodic data collection [2], is particularly relevant for applications where real-time information is crucial. Given that the data collected by IoT sensors often involve sensitive information tied to personal privacy [3], ensuring robust security is of paramount importance.

Authentication and key agreement (AKA) protocols are central to securing IoT systems, providing key features, such as anonymous authentication, privacy protection, confidentiality, integrity, and non-repudiation. These protocols rely on shared cryptographic keys to protect data. However, IoT systems often face unique security challenges. Many sensors are installed in unattended environments, making them vulnerable to physical attacks.

Furthermore, the data they collect are transmitted over wireless channels that are susceptible to interception, modification, and deletion by malicious actors [4,5]. Given the limited computational resources of many IoT devices, there is a pressing need for lightweight AKA protocols that can provide strong security guarantees while minimizing the computational and communication overheads [6].

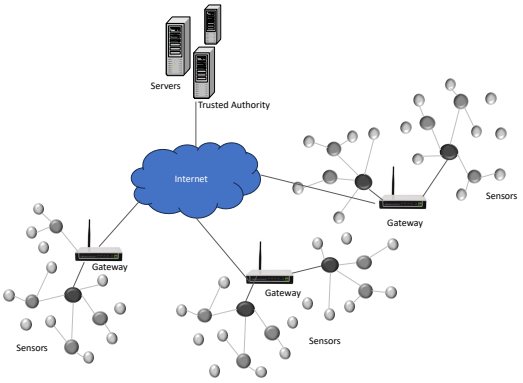

Figure 1. IoT network model.

Several IoT authentication schemes employ lightweight cryptographic techniques, such as symmetric encryption or hashing-based synchronization [7]. However, these approaches can expose systems to various vulnerabilities, including desynchronization attacks [8]. Public-key cryptography, particularly Elliptic Curve Cryptography (ECC), has been identified as a promising solution, offering strong security with smaller key sizes, thus reducing computational, storage, and bandwidth requirements. ECC has been incorporated into AKA protocols for IoT [9], providing a balance between security and efficiency. Nevertheless, designing ECC-based AKA protocols for IoT with a minimal overhead remains a complex challenge.

As shown in Table 1, several ECC-based authentication and key agreement (AKA) schemes have been proposed for IoT in recent years. To ensure anonymity, Tsai et al. [10] used temporary dynamic public keys to encrypt and decrypt the identity information of the sensor. Although this method improves anonymity, it also leads to an increased computational overhead. Similar trade-offs are observed in other schemes [11–23]. Furthermore, some protocols [23–29] do not provide sufficient anonymity and remain susceptible to identity leakage under specific conditions. Furthermore, certain schemes [14,18,20,22,23] risk compromising availability while trying to maintain anonymity. For example, when a sensor sends an authentication request or response without including its identity information, or if the protocol is inadequately designed, the server may be required to search through stored data and recompute the temporary public key to identify the sensor. Moreover, an AKA protocol must be resistant to ephemeral secret leakage (ESL) attacks, as ephemeral keys may be precomputed and stored in insecure memory [30]. However, schemes such as [10,11,16–18,21,23–25,27,31,32] do not effectively mitigate ESL attacks, leaving session keys vulnerable if ephemeral keys are compromised. Finally, perfect forward secrecy (PFS) ensures that even if an attacker gains access to a private key or long-term authentication information, past session keys remain secure and cannot be decrypted. However, existing schemes [11,13,17,24–26,28,33,34] do not provide PFS. Additionally, many of these protocols are either prone to other attacks or are not lightweight enough for resource-constrained IoT environments. In conclusion, while various ECC-based AKA schemes enhance the anonymity and security in IoT, many come with significant trade-offs, including an increased computational overhead, vulnerability to ESL attacks, and a lack of PFS. These limitations make them unsuitable for IoT environments that require both robust security and efficiency in resource-constrained settings.

Table 1. Summary of relevant ECC-based AKA protocols.

Scheme	Year	Shortcomings
[10]	2016	Prone to ESL attacks and high computation cost.
[23]	2016	Prone to DoS and ESL attacks, lacking anonymity, and poor availability.
[11]	2017	Prone to ESL attacks, lacking PFS, and high computation cost.
[33]	2017	Prone to IM attacks, lacking PFS, and high computation cost.
[24]	2017	Prone to ESL and private key leakage attacks and lacking PFS and anonymity.
[25]	2017	Prone to ESL attacks and lacking PFS, anonymity, and mutual authentication.
[12]	2018	Prone to MIM and KCI attacks.
[13]	2018	Prone to MIM and KCI attacks, lacking PFS, and the key control problem.
[14]	2019	Prone to DoS attacks and poor availability.
[26]	2019	Prone to replay and DoS attacks and lacking anonymity and mutual authentication.
[27]	2019	Prone to ESL attacks, lacking anonymity, and high computation cost.
[15]	2020	Prone to KCI attacks and high computation cost.
[31]	2020	Prone to ESL attacks and high computation cost.
[28]	2020	Prone to KCI attacks and lacking PFS and anonymity.
[16]	2021	Prone to KCI and ESL attacks.
[34]	2021	Lacking PFS and high computation cost.
[17]	2021	Prone to MIM, IM, DoS, and ESL attacks and lacking PFS.
[18]	2021	Prone to MIM, IM, and ESL attacks and poor availability.
[29]	2022	Lacking anonymity.
[19]	2022	Prone to MIM attacks.
[20]	2022	Prone to MIM and KCI attacks, lacking PFS, and poor availability.
[32]	2023	Prone to ESL attacks and high computation cost.
[21]	2023	Prone to ESL and desynchronization attacks.
[22]	2024	Prone to DoS attacks, high computation cost, and poor availability.

ESL: ephemeral secret leakage; KCI: key compromise impersonation; MIM: man-in-the-middle; IM: impersonation; PFS: perfect forward secrecy.

Inspired by fraud detection techniques [35,36] and the provision of perfect forward secrecy (PFS) [37,38] based on dynamic authenticated credentials (DACs), we propose a DAC synchronization framework for anonymous authentication. To ensure feasibility, we modified the DAC update strategy. To maintain synchronization between the server and the sensor, both update the DACs using the accepted session key. The server holds off on deleting the previous DAC until the next one is received from the sensor. This reduces the communication overhead to only two messages, matching the number of communications in the authentication and key negotiation process. If a sensor node uses the same DAC for repeated logins, it will be resynchronized by the server. To address potential inconsistencies in the DAC due to power interruptions or calculation errors, the server can resynchronize any unlogged sensors using a resynchronization procedure.

We then propose an ECC-based AKA scheme that utilizes DACs that reduce overhead and improves security. First, our approach uses DACs for anonymous authentication instead of temporary public keys or sender credentials. Moreover, the proposal provides PFS and resistance to ephemeral secret leakage (ESL) attacks since session keys are derived from both long-term and ephemeral credentials. Even if long-term or ephemeral keys from a single session are compromised, past and future session keys remain secure. Performance evaluations showed that our protocol reduced the communication costs by more than 5% and the computational overhead by more than 37%.

The contributions of this work are as follows:

(1) A literature survey that analyzes efficiency, anonymity, PFS, and vulnerabilities to ESL attacks in related schemes, identifying potential security concerns and weaknesses.
(2) A novel DAC synchronization framework integrated with a lightweight ECC-based anonymous AKA protocol. This framework reduced the computational overhead by more than 37% while improving security features and reducing communication overhead compared with existing schemes.
(3) Formal analysis and automatic verification of the proposal, which guaranteed mutual authentication, anonymity, PFS, and resistance to ESL attacks.

This paper is organized as follows: Section 2 introduces the network model and complexity assumptions. In Section 3, we present the proposed scheme. Following this, in Sections 4–7, we offer the security analysis results and performance comparison. Finally, in Section 8, we conclude this paper.

2. Preliminaries

2.1. Network Model

Based on the network models in recent schemes [20–22], the network consists of sensor nodes, gateways, and servers. As illustrated in Figure 1, sensor nodes and servers operate in periodic cycles determined by the frequency of data collection and monitoring requirements. Trusted Authorities (TAs) function as specialized entities responsible for key generation, distribution, management, and facilitating secure communication within the system. Both sensors and servers can register with various TAs, enabling them to join different systems. An IoT application may consist of one or more servers, depending on the architecture.

In our proposal, we assumed the involvement of S_s (where $1 \leq s \leq m$), SP_{sp} (where $1 \leq sp \leq n$), and TA_l (where $1 \leq l \leq z$), where $m \gg n, m \gg z$. Here, m, n, and z denote the respective numbers of sensors, servers, and TAs.

The operational cycles of sensor nodes and servers are influenced by factors such as the data collection frequency, environmental monitoring needs, and security requirements. For example, the cycles of the sensor nodes are shaped by the data collection rate, battery life, and the necessity for periodic updates. In contrast, server cycles are determined by the network load and the need to handle authentication processes or update credentials.

2.2. Complexity Assumptions

Consider a non-singular elliptic curve E defined over a finite field F_p, where p is a large prime. Let G be the additive group of points on E, with order q and base point P, where q is also a large prime number.

Definition 1. *Elliptic Curve Discrete Logarithm (ECDL) Harness Assumption: Given $\langle P, xP \in G \rangle$, finding $x \in \mathbb{Z}_q^*$ is computationally difficult. The probability that an algorithm $\mathcal{A}$ can solve this problem in time t is negligible for sufficiently small ϵ:*

$$\mathrm{Adv}_{\mathrm{ECDL}}^{\mathcal{A}}(t) = \Pr[\mathcal{A}(P, xP) = x : x \in \mathbb{Z}_q^*] < \epsilon \tag{1}$$

Definition 2. *Computational Diffie–Hellman (CDH) Hardness Assumption: Given $\langle xP, yP \in G \rangle$, computing $xyP \in G$ is computationally hard. The probability that an algorithm $\mathcal{A}$ can solve this problem in time t is negligible for sufficiently small ϵ:*

$$\mathrm{Adv}_{\mathrm{CDH}}^{\mathcal{A}}(t) = \Pr[\mathcal{A}(xP, yP) = xyP : x, y \in \mathbb{Z}_q^*] < \epsilon \tag{2}$$

2.3. Threat Model

The threat model is based on the eCK adversary model [39] and the state-of-the-art schemes [20–22]. The adversary $\mathcal{A}$ is assumed to possess the following capabilities:

T1. Channel control: $\mathcal{A}$ can completely control the communication channels, allowing them to intercept, modify, block, or delete messages exchanged between the server and the sensor.

T2. Session key compromise: $\mathcal{A}$ can capture session keys used in previous exchanges between the server and the sensor.

T3. Long-term secret exposure: $\mathcal{A}$ can compromise either the sensors or the server, gaining access to their long-term secrets.

T4. Ephemeral secret exposure: $\mathcal{A}$ has the ability to obtain ephemeral secrets from both the server and the sensor.

2.4. Evaluation Criteria

The evaluation criteria are based on current state-of-the-art schemes [20–22]. The criteria include the following:

E1. Mutual authentication and key agreement: The scheme must ensure mutual authentication between the user and the sensor, as well as the successful establishment of a shared session key.

E2. Perfect forward secrecy (PFS): The shared session key must remain secure and unrecoverable, even if the adversary captures the long-term secrets of the server, TA, or sensor.

E3. Anonymity: The scheme should protect the identities of the entities, prevent activity correlation, and ensure untraceability.

E4. Resistance to classic attacks: The protocol must withstand various classic attacks, such as KCI attacks, ephemeral secret leakage, unknown key share, known key, impersonation, desynchronization, replay, and IoT node capture attacks.

3. The Proposed Scheme

This section briefly describes the proposal, which includes system initialization and preloading, server and sensor registration, authentication and key agreement, and resynchronization. Table 2 shows some symbols.

Table 2. Notations used in the protocols.

Notation	Description
S	Sensor node
SP	Server
TA	Trusted third party
$\mathcal{A}$	Adversary $\mathcal{A}$
SSK	Session key
ID_i	Identity of protocol entity i
r_i,x_i	Temporary secret of entity i
k_i	Long-term keys of entity i
WT_i	Public key-based signature
TC_{ssp}	Dynamic authenticated credential of S to SP

3.1. System Initialization and Preloading

To select the system parameters, the TA follows these steps:

STA1: The TA chooses an elliptic curve $E(a,b)$ with a base point P over $\mathbb{Z}_q^* = \{0,1,\ldots,q-1\}$, where q is a large prime number.

STA2: The TA selects two one-way hash functions: $h : \{0,1\}^* \rightarrow \{0,1\}^l$ and $h_1 : \{0,1\}^* \rightarrow \{0,1\}^{128}$.

- Function $h(\cdot)$: This hash function is used to securely combine various pieces of information, such as session identifiers, public keys, and other relevant data, into a single value.
- Function $h_1(\cdot)$: This hash function is specifically utilized for the generation and updating of DACs.

STA3: Finally, the TA preloads the tuple $\{(E(a,b), q, P, h, h_1)\}$, along with its own identifier, to each sensor and server.

3.2. Registration

The registration process for each sensor node S and server SP with the TA is detailed in the following steps:

R1: First, S chooses a random number $r_s \in Z_q^*$. Next, S computes $R_s = r_s \cdot P$ and securely sends the registration request $\{ID_s, R_s\}$ to the TA.

Similarly, for server SP, SP selects a random number $r_{sp} \in Z_q^*$. Then, SP calculates $R_{sp} = r_{sp} \cdot P$ and securely transmits $\{ID_{sp}, R_{sp}\}$ to the TA.

R2: The TA selects a random number $r_{ta}^s \in Z_q^*$ for each valid S with ID ID_s. Then, the TA computes the public key of S using $PK_s = R_s + r_{ta}^s \cdot P$ and the DAC of S using $TC_{ssp} = h_1(ID_s \| r_{ta}^s)$.

For server SP, the TA selects a random number $r_{ta}^{sp} \in Z_q^*$ for each valid ID_{sp} and computes the public key $PK_{sp} = R_s + r_{ta}^{sp} \cdot P$. Then, the TA selects another random number X_{sp} for each valid SP.

R3: The TA stores $\{ID_s, PK_s, ID_{sp}, PK_{sp}, X_{sp}\}$ and then transmits $\{TC_{ssp}, PK_s, r_{ta}^s, ID_{sp}, PK_{sp}, X_{sp}\}$ to S and $\{PK_{sp}, r_{ta}^{sp}, X_{sp}, ID_s, TC_{ssp}, PK_s\}$ to SP through secure channels.

R4: Upon receiving the reply message, S calculates the private key $k_s = ((r_s + r_{ta}^s) \bmod q)$. Then, S checks whether $PK_s = k_s \cdot P$. If valid, S computes $WT_{sp} = k_s \cdot PK_{sp}$ and stores $\{ID_s, PK_s, k_s, TC_{ssp}, ID_{sp}, WT_{sp}, X_{sp}\}$.

Similarly, SP computes the private key $k_{sp} = ((r_{sp} + r_{ta}^{sp}) \bmod q)$ and computes $WT_s = k_{sp} \cdot PK_s$. Then, SP initializes the pre-DAC and S-login tokens, and sets $prTC_{ssp} = TC_{ssp}$ and $Login_s = Null$. Finally, SP stores $\{ID_{sp}, PK_{sp}, k_{sp}, X_{sp}, ID_s, prTC_{ssp}, TC_{ssp}, Login_s, WT_s\}$.

When a new or frozen sensor S' is registered, the TA securely transmits $\{ID_{s'}, TC_{s'sp}, PK_{s'}\}$ to the corresponding SP.

3.3. Authentication and Key Agreement

As shown in Figure 2, this subsection describes the mutual authentication and key agreement processes between S and SP without the help of the TA.

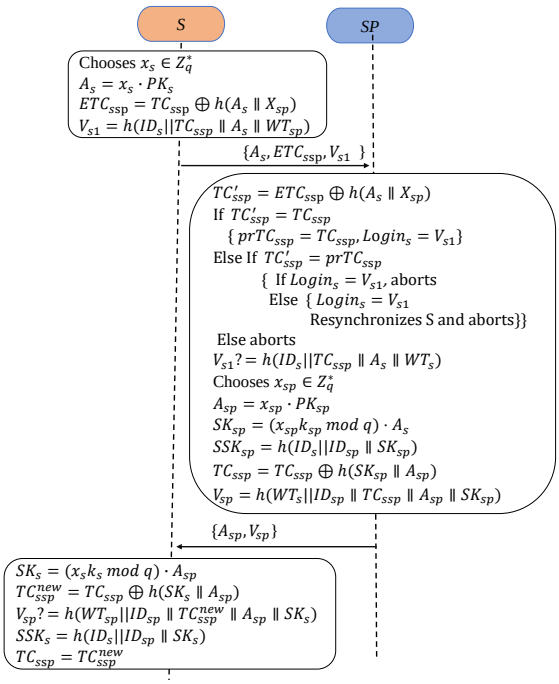

Figure 2. Authentication and key agreement.

A1: First, sensor S selects a random nonce $x_s \in Z_q^*$. Then, S computes $A_s = x_s \cdot PK_s$ and $ETC_{ssp} = TC_{ssp} \oplus h(A_s \| X_{sp})$. Next, S determines its verifier $V_{s1} = h(ID_s \| TC_{ssp} \| A_s \| WT_{sp})$. Finally, S sends the authentication request message $\{A_s, ETC_{ssp}, V_{s1}\}$ to the server SP.

A2: In response, SP computes $TC'_{ssp} = ETC_{ssp} \oplus h(A_s \| X_{sp})$ to verify the DAC received against TC'_{ssp}. If $TC'_{ssp} = TC_{ssp}$, then $prTC_{ssp} = TC_{ssp}$ and $Login_s = V_{s1}$ are set. Else, if $TC'_{ssp} = prTC_{ssp}$, SP processes as follows:

If $Login_s = V_{s1}$, SP terminates the session.

Otherwise, SP sets $Login_s = V_{s1}$, sends a resynchronization message to S, and terminates the session.

If neither condition holds, the session is aborted.

Finally, SP confirms the integrity of the incoming message and the validity of S by checking whether $V_s = h(ID_s \| TC_{ssp} \| A_s \| WT_s)$. If invalid, the session is aborted.

A3: First, server SP selects a random nonce $x_{sp} \in Z_q^*$ and computes $A_{sp} = x_{sp} \cdot PK_{sp}$. Then, SP computes $SK_{sp} = ((x_{sp} k_{sp} \mod q) \cdot A_s)$, which is used to compute the session key as $SSK_{sp} = h(ID_s \| ID_{sp} \| SK_{sp})$. Next, SP computes a new DAC for S: $TC_{ssp} = TC_{ssp} \oplus h(SK_{sp} \| A_{sp})$. After this, SP obtains its verifier $V_{sp} = h(WT_s \| ID_{sp} \| TC_{ssp} \| A_{sp} \| SSK_{sp})$. Finally, SP transmits the reply message $\{A_{sp}, V_{sp}\}$ to S.

A4: In response, S first computes $SK_s = ((x_s k_s \mod q) \cdot A_{sp})$ and generates a new temporal credential $TC_{ssp}^{new} = TC_{ssp} \oplus h(SK_s \| A_{sp})$. Next, S verifies the equivalence of $V_{sp} = h(WT_s \| ID_{sp} \| TC_{ssp}^{new1} | A_{sp} \| SSK_s)$. If the verification is satisfied, S determines the session key using $SSK_s = h(ID_s \| ID_{sp} \| SK_s)$ and updates TC_{ssp} with TC_{ssp}^{new}.

Remark 1. *When there are storage or computation errors, power-down interruptions, etc., these would cause the TC_{ssp} maintained by S to be inconsistent with either TC_{ssp} or $prTC_{ssp}$ on the server side, which then prevents S from logging into SP. SP will resynchronize the unlogged sensors before the end of the collaboration cycle, following the resynchronization procedures.*

3.4. Re-Synchronization

SR1: First, server SP selects a random nonce $y_s \in Z_q^*$ and then computes $TC_{ssp} = prTC_{ssp} = h1(ID_s \| y_s)$, $MTC_{ssp} = TC_{ssp} \oplus h(WT_s \| X_{sp})$, and $MV_{sp} = h(ID_{sp} \| ID_s \| TC_{ssp} \| WT_s)$. Finally, SP transmits the resynchronization message $\{MTC_{ssp}, MV_{sp}, Resyn\}$ to S.

SR2: In response, S computes $TC_{ssp}^{new} = MTC_{ssp} \oplus h(WT_{sp} \| X_{sp})$, and then checks whether $MV_{sp} = h(ID_{sp} \| ID_s \| TC_{ssp}^{new} \| WT_{sp})$. If the verification is satisfied, S sets $TC_{ssp} = TC_{ssp}^{new}$.

4. Formal Security Proof

This section discusses the security of the proposal within the eCK security model [39].

4.1. Participants

The proposal involves two participants: a sensor S and a server SP. These participants may have multiple oracles, each participating in separate concurrent executions of the protocol. The oracles for S and SP are denoted by S_i and SP_j, where $i, j \in \mathbb{Z}$, and any oracle is represented by $I \in S \cup SP$. Each oracle has a session identifier Sid, which represents the messages it sends and receives. There are three possible states for an oracle:

Accept: The oracle reaches this state after receiving the expected message in the protocol.

Reject: The oracle enters this state if an unexpected or incorrect message is received.

$\perp$: If the oracle does not respond to input, it transitions to this state.

4.2. Adversary

Based on threat models, the adversary $\mathcal{A}$ controls the public channels and operates in a probabilistic polynomial-time (PPT) manner. $\mathcal{A}$ has the ability to inspect, modify, verify, and insert information into communication between oracles. Furthermore, $\mathcal{A}$ can gain knowledge of the secret credentials of the participants and can interact with the oracles S_i and SP_j using the following queries:

(1) $Execute(SM_i, SP_j)$: $\mathcal{A}$ can obtain the messages $\{A_s, TC_{ssp}, V_{s1}\}$ and $\{A_{sp}, V_{sp}\}$ using this query.
(2) $Send(I, m_I)$: $\mathcal{A}$ can send a message m_I to oracle I and receive a reply from I according to the protocol.
(3) $ESReveal(I)$: This query allows $\mathcal{A}$ to learn the ephemeral secrets of the oracle I.
(4) $SKReveal(I)$: $\mathcal{A}$ can reveal the session key owned by oracle I using this query.
(5) $Corrupt(I)$: $\mathcal{A}$ can obtain long-term confidential information held by the oracle I.
(6) $h(m)$: This query allows $\mathcal{A}$ to obtain a random hash output for message m.
(7) $Test(I)$: This query tests the semantic security of the session key SSK_s or SSK_{sp}. If the session key has not been established, the query returns $\perp$. Otherwise, a private coin d is flipped:
   - If $d = 1$, the session key SSK_s or SSK_{sp} is returned to $\mathcal{A}$.
   - If $d = 0$, a random key of the same size is returned. The goal of $\mathcal{A}$ is to distinguish between real session keys and random numbers.
(8) $Expire(I)$: This query removes the session key held by oracle I.

4.3. Session Definition

Fresh: A session *se* held by oracle I is considered fresh if it and its partner session have not been revealed through queries SK Reveal(I), ESReveal(I), or Corrupt(I) before the session expires with query Expire(I), even if the session has not yet formed.

Partners: Two oracles, S_i and SP_j, are considered partners if the following conditions are met: (1) Both oracles reach the accept state. (2) Both oracles are fresh. (3) They authenticate each other and agree on a session key using the same session identifier *Sid*.

Semantic security: In the game $\text{Game}_{\text{AKA}}(I, \mathcal{A})$, the security of session keys SSK_s or SSK_{sp} is simulated. The adversary $\mathcal{A}$ may issue a single Test(I) query, and a bit d' is returned if the oracle I has reached the accept state and is fresh. Although only one Test(I) query is allowed, $\mathcal{A}$ can still make multiple other queries to the oracle I. Let $\Pr[\text{Succ}(\mathcal{A})]$ denote the probability that $\mathcal{A}$ succeeds in winning the game $\text{Game}_{\text{AKA}}(I, \mathcal{A})$. The advantage of $\mathcal{A}$ in breaching the proposal's semantic security is defined as

$$\text{Adv}(\mathcal{A}) = \left|2 \cdot \Pr\left[d' = d\right] - 1\right| = \left|2 \cdot \Pr[\text{Succ}(\mathcal{A})] - 1\right| \tag{3}$$

Definition 3. *Under the eCK adversary model, an AKA scheme is considered semantically secure if the advantage of the adversary $\mathcal{A}$, $\text{Adv}(\mathcal{A})$, is bounded by a sufficiently small value ϵ, that is, $\text{Adv}(\mathcal{A}) \leq \epsilon$.*

4.4. Formal Security Analysis

Theorem 1. *Assume that to breach the semantic security of the proposed scheme, the adversary $\mathcal{A}$ can perform at most q_e Execute( ) queries, q_s Send() queries, and $q_h h()$ queries within a time limit t. Given that the hash output length is l, the advantage of $\mathcal{A}$ is bounded by the following:*

$$\begin{aligned} Adv(\mathcal{A}) \leq & \frac{(q_h^2 + 2q_s)}{2^l} + \frac{(q_s + q_e)^2}{2(q-1)} \\ & + \left(\frac{3q_s}{q-1} + \frac{3q_s}{2^l}\right) \max\{Adv^{\mathcal{A}}_{\text{ECDL}}(t), Adv^{\mathcal{A}}_{\text{ECD}}(t)\} \end{aligned} \tag{4}$$

Proof. We define a series of six games GM_i for $i = 0, 1, \ldots, 5$ to aid in proving the theorem. When $\mathcal{A}$ correctly guesses the bit d returned by the $Test(I)$ query, the event S_i occurs for each game.

GM_0: In this game, we simulate an actual attack in the random oracle model (ROR). The advantage in this game is the difference between the probability that the adversary succeeds and 0.5:

$$Adv(A) = \mid 2Pr[S_0] - 1 \mid \tag{5}$$

GM_1: Here, $\mathcal{A}$ can use the Execute(S_i, SP_j) query to obtain messages such as $\{A_s, TC_{ssp}, V_{s1}\}$ and $\{A_{sp}, V_{sp}\}$. Next, $\mathcal{A}$ checks the authenticity of the session key SSK_s (or SSK_{sp}) using SK Reveal(I) and Test(I) queries. The messages exchanged during the authentication process do not provide enough information to infer the session key. Therefore, distinguishing between an actual attack and Game GM_1 is not possible:

$$Pr[S_1] = Pr[S_0] \tag{6}$$

Three lists are used to track the results of various queries:

- L_h: The inputs and outputs of the queries $h()$.
- $L_h^{\mathcal{A}}$: The answers to the $h()$ queries asked by $\mathcal{A}$.
- L_E: The inputs and outputs of the Execute( ) queries.

GM_2: This game simulates that $\mathcal{A}$ might forge messages accepted by queries $Sen()$ and $h()$. Semantic security can only be compromised if $\mathcal{A}$ detects collisions and sends a valid message. The probability of collisions in the $h()$ outputs is bounded by the birthday paradox $\frac{q_h^2}{2^{l+1}}$. Furthermore, the probability of collisions in random numbers is at most $\frac{(q_s+q_e)^2}{2(q-1)}$. As a result, it is impossible to distinguish between GM_2 and GM_1 unless the mentioned collisions occur:

$$|Pr[S_2] - Pr[S_1]| \leq \frac{(q_s + q_e)^2}{2(q-1)} + \frac{q_h^2}{2^{l+1}} \tag{7}$$

GM_3: In this game, we simulate the case where $\mathcal{A}$ is able to guess certain parameters and forge messages, such as $\{A_s, TC_{ssp}, V_{s1}\}$ and $\{A_{sp}, V_{sp}\}$ without using random oracles. If validation fails, the queries are terminated. The probabilities for Send(A_s, TC_{ssp}, V_{s1}) and Send(A_{sp}, V_{sp}) are each bounded by $\frac{q_s}{2}$. With these additional operations, Games GM_3 and GM_2 are indistinguishable:

$$|Pr[S_3] - Pr[S_2]| \leq \frac{2q_s}{2^l} \tag{8}$$

GM_4: This game models the corruption capability of $\mathcal{A}$.$\mathcal{A}$ so that it cannot learn SSK_s or SSK_{sp} unless it captures (x_s, k_s) or (x_{sp}, k_{sp}). The adversary can attempt to retrieve x_s and x_{sp} by issuing Execute() and $h()$ queries. In the following cases, the adversary may retrieve information about the session keys:

(1) If $\mathcal{A}$ corrupts both the oracles S_i and SP_j, the attack succeeds with probability $\frac{2q_s}{q-1}$.
(2) If $\mathcal{A}$ issues $Corrupt(S_i)$ and $ESReveal(SP_j)$, the probability of success is $\frac{q_s}{2^2} + \frac{q_s}{q-1}$.
(3) If $\mathcal{A}$ issues $ESReveal(S_i)$ and $Corrupt(SP_j)$, the probability is $\frac{q_s}{q-1} + \frac{q_{s'}}{2^2}$.
(4) If both $ESReveal(S_i)$ and ESReveal(SP_j) are issued, the success probability is $\frac{2q_s}{2}$.

In all these cases, $\mathcal{A}$ cannot determine the session keys unless it solves the ECDL or ECD problem within the time limit t:

$$|Pr[S_4] - Pr[S_3]| \leq (\frac{3q_s}{q-1} + \frac{3q_s}{2^l}) \max\{Adv_{\text{ECDL}}^{\mathcal{A}}(t), Adv_{\text{ECD}}^{\mathcal{A}}(t)\} \tag{9}$$

GM_5: Game GM_5 simulates GM_4 except that the Test() query is terminated if $\mathcal{A}$ issues an $h(ID_s \| ID_{sp} \| SSK_s)$ query. The maximum probability for $\mathcal{A}$ to retrieve SSK_s is $\frac{q_n^2}{2^t}$. Hence, SSK_s is $q_h^2/2^l$. Hence,

$$|Pr[S_5] - Pr[S_4]| \leq q_h^2/2^{l+1} \tag{10}$$

Without the correct input for the $h()$ query, $\mathcal{A}$ cannot differentiate between the actual session key and a random one:

$$Pr[S_5] = 1/2 \tag{11}$$

Finally, combining all the probabilities, we conclude that Theorem 1 holds true. □

5. Automatic Verification Using ProVerif

ProVerif [40] is a widely recognized tool for verifying the security properties of cryptographic protocols, particularly in the context of secrecy, authentication, and resistance to attacks. It uses *pi* calculus, a process algebra, and Prolog rules to assess the confidentiality of protocols. ProVerif supports a wide range of cryptographic primitives, including Diffie–Hellman key exchanges, hash functions, and symmetric and asymmetric cryptographic algorithms.

In this section, we formally validate the proposed scheme using ProVerif. Table 3 presents the code for a sensor node during the registration and authentication phases. In this setup, *schst* represents a private communication channel between the sensor S and the TA during registration. The public channel *chsp* facilitates communication between the sensor S and the server SP during authentication. Similarly, each SP and TA has their own corresponding processes. The proposed protocol runs in parallel as $((!Sensor)(!Server)(!TA))$.

As shown in Table 4, the results show that $\mathcal{A}$ cannot decrypt the authentication process or obtain the long-term, ephemeral, and session keys. The proposal offers anonymity and consistency.

Table 3. Codes for sensor.

```
let Sensor=
new rs:bitstring;
let Rs= Mul(rs, P) in
out (schst, (IDs, Rs));
in (schst, (uTCssp:bitstring,uPKs: bitstring,ursta:bitstring, uIDsp:bitstring,uPKsp: bitstring));
let ks=Add (rs,ursta) in
let PKs=Mul (ks, P) in
if PKs=uPKs then
let WTsp= Mul (ks, uPKsp) in
!
(
event startAuthsp;
let As= Mul(xs,PKs) in
let ETCssp= H(Con(uTCssp,Con (As, Xsp))) in let Vs1= H(Con (Con (Con (IDs, uTCssp), As),
WTsp))in
out (chsp, (As, ETCssp, Vs1));
in (chsp, (sAsp: bitstring,sVsp:bitstring));
let SKs= Mul (Mul (xs, ks), sAsp) in
let TCsspnew= Xor (TCssp, H(Con(SKs,sAsp))) in
let uVsp= H(Con(Con (Con (Con (WTsp,IDsp),
   TCsspnew), sAsp), SKs)) in
if uVsp= sVsp then
let ssks=H(con(con(IDs,IDsp),SKs)) in
let TCssp=TCsspnew in event endAuths;
0
).
```

Table 4. Simulation results.

Result	Target
RESULT Weak secret IDs is true (bad not derivable)	Anonymity
RESULT Weak secret IDsp is true (bad not derivable)	Anonymity
RESULT not attacker(ks[]) is true	Long-term secret security
RESULT not attacker(ksp[]) is true	Long-term secret security

Table 4. *Cont.*

Result	Target
RESULT not attacker(xs[]) is true	Ephemeral secret security
RESULT not attacker(xsp[]) is true	Ephemeral secret security
RESULT not attacker (SSKs[]) is true	Session key security
RESULT not attacker (SSKsp[]) is true	Session key security
RESULT inj-event(endAuths) ⇒ inj-event(startAuths) is true	Consistent
RESULT inj-event(endAuthsp) ⇒ inj-event(startAuthsp) is true	Consistent
RESULT inj-event(endAuthsp) ⇒ inj-event(endAuths) is true	Consistent
RESULT inj-event(endAuths) ⇒ inj-event(endAuthsp) is true	Consistent

6. Descriptive Security Analysis

6.1. Perfect Forward Secrecy

For $SSK_s = h(ID_s \| ID_{sp} \| SK_s)$, where $SK_s = ((x_s k_s) \bmod q) \cdot A_{sp}$, if an adversary $\mathcal{A}$ has obtained long-term secrets k_s, k_{sp}, WT_s, and WT_{sp}, it cannot access the random numbers x_p and x_{sp} for each session. This prevents $\mathcal{A}$ from determining SSK_s, ensuring perfect forward secrecy.

6.2. Impersonation Attacks Resistance

We first consider impersonation attacks targeting S. In this scenario, the adversary does not have access to long-term secrets $k_s, k_{sp}, TC_{ssp}, WT_s$, and WT_{sp}. Without these secrets, $\mathcal{A}$ cannot generate a valid authentication request message $\{A_s, TC_{ssp}, V_{s1}\}$, where V_{s1} includes WT_{sp} . As a result, $\mathcal{A}$ cannot impersonate S. Similarly, the adversary cannot impersonate SP without the corresponding secrets.

6.3. IoT Node Capture Attacks Resistance

While $\mathcal{A}$ may capture a sensor, it can only extract long-term credentials, such as $\{ID_s, PK_s, k_s, TC_{ssp}, ID_{sp}, WT_{sp}\}$. Since each sensor has unique credentials, the adversary can only compromise the session key between the captured sensor and SP. The session key security remains intact for any uncompromised sensor S'_s.

6.4. Key Compromise Impersonation Attack Resistance

Even if $\mathcal{A}$ gains access to S's credentials, it cannot impersonate SP or authenticate with S. This is because $\mathcal{A}$ does not have access to the necessary random numbers x_s and x_{sp}, which are crucial to the verification process. The credentials for S are based on $SSK_s = h(ID_s \| ID_{sp} \| SK_s)$, where $SK_s = ((x_s k_s) \bmod q) \cdot A_{sp}$ and $A_{sp} = x_{sp} \cdot PK_{sp}$.

6.5. Desynchronization Attacks Resistance

The scheme uses DACs and random numbers to protect against replay attacks and preserve user anonymity. The server prevents the deletion of prDAC to ensure that both parties share the same DAC, even if the sensor does not receive the authentication response from the server. Since participants do not need to synchronize their clocks, the scheme is resilient to desynchronization attacks.

7. Performance Comparison

This section compares the computational and communication costs of the proposed scheme with those of related schemes [19–22,29], as well as their security and functional attributes.

7.1. Computation Cost

Let T_{pm}, T_{pa}, T_h, and T_{sk} represent the running times for point multiplication, point addition, hash operations, and symmetric key operations, respectively. The server used for the evaluation is a laptop with an Intel Core i5 2 GHz processor, 16 GB RAM, and macOS 13.4.1. For an elliptic curve with $p = 2^{192}$, specifically $Curve25519$, and a point length of 384 bits, the average running times on this server were 1.258 ms for point multiplication,

0.006 ms for point addition, 0.005 ms for hash operations, and 0.007 ms for symmetric key operations. The sensor node used for testing was a Raspberry Pi 3 Model B+ board that featured an ARMCortex-A53 1.4 GHz processor and 1 GB RAM. Under the same testing conditions, the corresponding running times on the sensor node were 2.225 ms for point multiplication, 0.025 ms for point addition, 0.019 ms for hash operations, and 0.035 ms for symmetric key operations.

As shown in Table 5, the proposed scheme consistently outperformed the related schemes in terms of computational efficiency, with reductions that ranged from approximately 37.98% to 50%.

Table 5. Computation costs.

Scheme	Sensor (ms)	Server (ms)	Total (ms)	Decline (in %)
Ours	$2T_{pm} + 5T_h \approx 7.656$	$2T_{pm} + 5T_h \approx 2.541$	10.197	-
[29]	$3T_{pm} + 4T_h \approx 11.372$	$4T_{pm} + 2T_{pa} + 5T_h \approx 5.069$	16.441	37.98%
[19]	$4T_{pm} + 4T_h \approx 15.12$	$4T_{pm} + 4T_h \approx 5.052$	20.172	49.45%
[20]	$4T_{pm} + 7T_h \approx 15.216$	$4T_{pm} + 7T_h \approx 5.067$	20.283	49.73%
[21]	$3T_{pm} + 4T_h + 2T_{sk} \approx 11.49$	$4T_{pm} + 4T_h + 3T_{sk} \approx 5.073$	16.563	38.44%
[22]	$4T_{pm} + 6T_h + 2T_k \approx 15.312$	$4T_{pm} + 6T_h + 2T_k \approx 5.082$	20.394	50.00%

7.2. Communication Cost

Assuming that ID, TS, R, H, and G represent an identity, timestamp, random number, hash output, and length of an ECC point, respectively, their corresponding lengths are 128 bits, 32 bits, 256 bits, 256 bits, and 384 bits.

As shown in Table 6, the proposed scheme consistently achieved a notable reduction in communication cost in all comparisons, with reductions that ranged from 5.56% to 38.18% when compared with related schemes.

Table 6. Communication costs.

Scheme	Sensor	Server	Total	Decline (in %)
Ours	G + H + ID	G + H	1088 bits	-
[29]	2G + 2H + TS	G + H	1568 bits	30.61%
[19]	G + H + T + ID	G + H + TS	1152 bits	5.56%
[20]	G + 2H + TS	G + 2H + ID	1376 bits	20.93%
[21]	G + H + R + 2TS + 2ID	G + H + 2TS + ID	1472 bits	26.09%
[22]	2TS + 2H + G + ID	TS + ID + H + G	1760 bits	38.18%

7.3. Performance Comparison

We based our evaluation criteria on the schemes in [19–22,29]. These criteria are divided into two main categories: ideal attributes and security requirements. The ideal attributes were evaluated from a functional perspective, where we examined whether the scheme possessed the desired features. In contrast, security requirements were evaluated from an attack perspective, which focused on whether the adversary $\mathcal{A}$ could successfully break the scheme.

As shown in Table 7, the proposed protocol addressed the security vulnerabilities found in recent schemes, such as the lack of anonymity [29]; the absence of PFS [20]; the poor availability [19,22]; and susceptibility to attacks, such as man-in-the-middle [19,20], ESL [21], desynchronization [21], and DoS [22]. In conclusion, the proposed protocol provided the highest level of security and effectively met the functional and security requirements of IoT environments.

Table 7. Performance comparison.

Scheme	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13
[29]	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
[19]	✓	✓	✓	✓	✓	✓	✓	✓	✓	×	✓	✓	✓
[20]	✓	×	✓	×	✓	✓	✓	✓	✓	×	✓	×	✓
[21]	✓	✓	✓	✓	×	✓	×	✓	✓	✓	✓	✓	✓
[22]	✓	✓	✓	×	✓	×	✓	✓	✓	✓	✓	✓	✓
Ours	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

F1: anonymity; F2: PFS; F3: mutual authentication without RC/TA; F4: availability; F5: ESL resistance; F6: DoS attack resistance; F7: desynchronization attack resistance; F8: IoT node capture attacks resistance; F9: impersonation attack resistance; F10: man-in-the-middle attack resistance; F11: replay attack resistance; F12: key comprise impersonation attack resistance; F13: unknown key share attack resistance; ✓: a functional feature or security is supported by the scheme; ×: a feature is not supported or a scheme is insecure.

The performance analysis led to the following conclusions: In terms of efficiency, the proposed protocol demonstrated lower computation and communication overheads compared with recent schemes. It consistently outperformed related protocols in computational efficiency, where it achieved reductions that ranged from approximately 37.98% to 50%. In addition, the proposed scheme also achieved significant reductions in communication costs, with improvements that ranged from 5.56% to 38.18% compared with other schemes. Regarding security, our protocol offered comprehensive security properties and effectively mitigated known attacks. It satisfied the security requirements of IoT applications, which ensured robust protection against a wide range of threats.

8. Conclusions

We examined existing security schemes and discovered that many use shared temporary public keys to conceal IDs or validate sender certificates, resulting in an increased computational overhead. Additionally, many of these schemes have not covered the necessary security features, such as the lack of PFS and vulnerability to ESL attacks. To overcome these issues, we modified a new DAC framework and developed the proposed protocol with DACs. After the formal analysis and automated verification, we determined that the proposal allowed for mutual authentication, anonymity, and PFS, and was resistant to common attacks, such as ESL, KCI, and MIM. The performance analysis showed that the proposal offered more security features, required fewer communication overheads, and reduced the computation overhead by more than 37%.

Author Contributions: Conceptualization, S.H.; methodology, S.H. and M.L.; software, S.H.; validation, S.H.; formal analysis, S.H.; investigation, S.H.; resources, S.H.; data curation, S.H. and M.L.; writing—original draft preparation, S.H. and M.L.; writing—review and editing, S.H. and M.L.; visualization, S.H.; supervision, S.H.; project administration, S.H.; funding acquisition, S.H. All authors read and agreed to the published version of this manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Gurewitz, O.; Shifrin, M.; Dvir, E. Data Gathering Techniques in WSN: A Cross-Layer View. *Sensors* **2022**, *22*, 2650. [CrossRef]
2. Vangala, A.; Das, A.K.; Mitra, A.; Das, S.K.; Park, Y. Blockchain-Enabled Authenticated Key Agreement Scheme for Mobile Vehicles-Assisted Precision Agricultural IoT Networks. *IEEE Trans. Inf. Forensics Secur.* **2023**, *18*, 904–919. [CrossRef]

3. Wang, D.; Wang, P. Two Birds with One Stone: Two-Factor Authentication with Security Beyond Conventional Bound. *IEEE Trans. Dependable Secur. Comput.* **2018**, *15*, 708–722. [CrossRef]
4. Zhang, R.; Cheng, L.; Wang, S.; Lou, Y.; Gao, Y.; Wu, W.; Ng, D.W.K. Integrated Sensing and Communication with Massive MIMO: A Unified Tensor Approach for Channel and Target Parameter Estimation. *IEEE Trans. Wirel. Commun.* **2024**, *23*, 8571–8587. [CrossRef]
5. Ma, Y.; Ma, R.; Lin, Z.; Zhang, R.; Cai, Y.; Wu, W.; Wang, J. Improving Age of Information for Covert Communication With Time-Modulated Arrays. *IEEE Internet Things J.* **2024**, 1. [CrossRef]
6. Wang, D.; Li, W.; Wang, P. Measuring Two-Factor Authentication Schemes for Real-Time Data Access in Industrial Wireless Sensor Networks. *IEEE Trans. Ind. Inform.* **2018**, *14*, 4081–4092. [CrossRef]
7. Challa, S.; Das, A.K.; Gope, P.; Kumar, N.; Wu, F.; Vasilakos, A.V. Design and analysis of authenticated key agreement scheme in cloud-assisted cyber–physical systems. *Future Gener. Comput. Syst.* **2020**, *108*, 1267–1286. [CrossRef]
8. Halevi, S.; Krawczyk, H. Public-Key Cryptography and Password Protocols. In Proceedings of the 5th ACM Conference on Computer and Communications Security—CCS '98, New York, NY, USA, 2–5 November 1998; pp. 122–131. [CrossRef]
9. Szczechowiak, P.; Oliveira, L.B.; Scott, M.; Collier, M.; Dahab, R. NanoECC: Testing the Limits of Elliptic Curve Cryptography in Sensor Networks. In *Proceedings of the Wireless Sensor Networks*; Verdone, R., Ed.; Springer: Berlin/Heidelberg, Germany, 2008; pp. 305–320.
10. Tsai, J.L.; Lo, N.W. Secure Anonymous Key Distribution Scheme for Smart Grid. *IEEE Trans. Smart Grid* **2016**, *7*, 906–914. [CrossRef]
11. He, D.; Zeadally, S.; Kumar, N.; Lee, J.H. Anonymous Authentication for Wireless Body Area Networks with Provable Security. *IEEE Syst. J.* **2017**, *11*, 2590–2601. [CrossRef]
12. Mo, J.; Hu, Z.; Lin, Y. Remote user authentication and key agreement for mobile client–server environments on elliptic curve cryptography. *J. Supercomput.* **2018**, *74*, 5927–5943. [CrossRef]
13. Shen, J.; Gui, Z.; Ji, S.; Shen, J.; Tan, H.; Tang, Y. Cloud-aided lightweight certificateless authentication protocol with anonymity for wireless body area networks. *J. Netw. Comput. Appl.* **2018**, *106*, 117–123. [CrossRef]
14. Kumar, N.; Aujla, G.S.; Das, A.K.; Conti, M. ECCAuth: A Secure Authentication Protocol for Demand Response Management in a Smart Grid System. *IEEE Trans. Ind. Inform.* **2019**, *15*, 6572–6582. [CrossRef]
15. Abbasinezhad-Mood, D.; Ostad-Sharif, A.; Nikooghadam, M.; Mazinani, S.M. A Secure and Efficient Key Establishment Scheme for Communications of Smart Meters and Service Providers in Smart Grid. *IEEE Trans. Ind. Inform.* **2020**, *16*, 1495–1502. [CrossRef]
16. Kumar, M.; Chand, S. A Lightweight Cloud-Assisted Identity-Based Anonymous Authentication and Key Agreement Protocol for Secure Wireless Body Area Network. *IEEE Syst. J.* **2021**, *15*, 2779–2786. [CrossRef]
17. Srinivas, J.; Das, A.K.; Li, X.; Khan, M.K.; Jo, M. Designing Anonymous Signature-Based Authenticated Key Exchange Scheme for Internet of Things-Enabled Smart Grid Systems. *IEEE Trans. Ind. Inform.* **2021**, *17*, 4425–4436. [CrossRef]
18. Tsobdjou, L.D.; Pierre, S.; Quintero, A. A New Mutual Authentication and Key Agreement Protocol for Mobile Client—Server Environment. *IEEE Trans. Netw. Serv. Manag.* **2021**, *18*, 1275–1286. [CrossRef]
19. Chaudhry, S.A.; Nebhan, J.; Yahya, K.; Al-Turjman, F. A Privacy Enhanced Authentication Scheme for Securing Smart Grid Infrastructure. *IEEE Trans. Ind. Inform.* **2022**, *18*, 5000–5006. [CrossRef]
20. Hajian, R.; Haghighat, A.; Erfani, S. A Secure Anonymous D2D Mutual Authentication and Key Agreement Protocol for IoT. *Internet Things* **2022**, *18*, 100493. [CrossRef]
21. Chaudhry, S.A.; Yahya, K.; Garg, S.; Kaddoum, G.; Hassan, M.M.; Zikria, Y.B. LAS-SG: An Elliptic Curve-Based Lightweight Authentication Scheme for Smart Grid Environments. *IEEE Trans. Ind. Inform.* **2023**, *19*, 1504–1511. [CrossRef]
22. Wu, Y.; Guo, H.; Han, Y.; Li, S.; Liu, J. A Security-Enhanced Authentication and Key Agreement Protocol in Smart Grid. *IEEE Trans. Ind. Inform.* **2024**, *20*, 11449–11457. [CrossRef]
23. He, D.; Wang, H.; Khan, M.K.; Wang, L. Lightweight anonymous key distribution scheme for smart grid using elliptic curve cryptography. *IET Commun.* **2016**, *10*, 1795–1802. [CrossRef]
24. Mahmood, K.; Chaudhry, S.A.; Naqvi, H.; Kumari, S.; Li, X.; Sangaiah, A.K. An elliptic curve cryptography based lightweight authentication scheme for smart grid communication. *Future Gener. Comput. Syst.* **2018**, *81*, 557–565. [CrossRef]
25. Abbasinezhad-Mood, D.; Nikooghadam, M. Design and hardware implementation of a security-enhanced elliptic curve cryptography based lightweight authentication scheme for smart grid communications. *Future Gener. Comput. Syst.* **2018**, *84*, 47–57. [CrossRef]
26. Li, Q.; Hsu, C.F.; Raymond Choo, K.K.; He, D. A Provably Secure and Lightweight Identity-Based Two-Party Authenticated Key Agreement Protocol for Vehicular Ad Hoc Networks. *Secur. Commun. Netw.* **2019**, *2019*, 7871067. [CrossRef]
27. Saeed, M.; Liu, Q.Y.; Tian, G.Y.; Gao, B.; Li, F. AKAIoTs: Authenticated key agreement for Internet of Things. *Wirel. Netw.* **2019**, *25*, 3081–3101. [CrossRef]
28. Garg, S.; Kaur, K.; Kaddoum, G.; Rodrigues, J.J.P.C.; Guizani, M. Secure and Lightweight Authentication Scheme for Smart Metering Infrastructure in Smart Grid. *IEEE Trans. Ind. Inform.* **2020**, *16*, 3548–3557. [CrossRef]
29. Yang, X.; Yi, X.; Nepal, S.; Khalil, I.; Huang, X.; Shen, J. Efficient and Anonymous Authentication for Healthcare Service With Cloud Based WBANs. *IEEE Trans. Serv. Comput.* **2022**, *15*, 2728–2741. [CrossRef]

30. Krawczyk, H. HMQV: A high-performance secure diffie-hellman protocol. In Proceedings of the Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), Santa Barbara, CA, USA, 14–18 August 2005; Springer: Berlin/Heidelberg, Germany, 2006; Volume 3621, pp. 546–566.
31. Harmanpreet Singh, G.; Adarsh; Devender, K. Cryptanalysis and improvement of a three-factor user authentication scheme for smart grid environment. *J. Reliab. Intell. Environ.* **2020**, *6*, 249–260. [CrossRef]
32. Cong, W.; Peng, H.; Maode, M.; Tong, Z.; Yiying, Z. A provable secure and lightweight ECC-based authenticated key agreement scheme for edge computing infrastructure in smart grid. *Computing* **2023**, *105*, 2511–2537. [CrossRef]
33. Wazid, M.; Das, A.K.; Kumar, N.; Rodrigues, J.J.P.C. Secure Three-Factor User Authentication Scheme for Renewable-Energy-Based Smart Grid Environment. *IEEE Trans. Ind. Inform.* **2017**, *13*, 3144–3153. [CrossRef]
34. Chaudhry, S.A.; Alhakami, H.; Baz, A.; Al-Turjman, F. Securing Demand Response Management: A Certificate-Based Access Control in Smart Grid Edge Computing Infrastructure. *IEEE Access* **2020**, *8*, 101235–101243. [CrossRef]
35. Nali, D.; van Oorschot, P.C. CROO: A Universal Infrastructure and Protocol to Detect Identity Fraud. In *Computer Security—ESORICS 2008*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2008; Volume 5283, pp. 130–145.
36. Yang, Z.; Guo, F. Authenticated key exchange with synchronized state. *Secur. Commun. Netw.* **2014**, *7*, 2373–2388. [CrossRef]
37. Yang, Z.; Lai, J.; Sun, Y.; Zhou, J. A Novel Authenticated Key Agreement Protocol With Dynamic Credential for WSNs. *ACM Trans. Sen. Netw.* **2019**, *15*, 1–27. [CrossRef]
38. Yang, Z.; He, J.; Tian, Y.; Zhou, J. Faster Authenticated Key Agreement with Perfect Forward Secrecy for Industrial Internet-of-Things. *IEEE Trans. Ind. Inform.* **2020**, *16*, 6584–6596. [CrossRef]
39. LaMacchia, B.; Lauter, K.; Mityagin, A. Stronger Security of Authenticated Key Exchange. In Proceedings of the Provable Security, Wollongong, Australia, 1–2 November 2007; Susilo, W., Liu, J.K., Mu, Y., Eds.; Springer: Berlin/Heidelberg, Germany, 2007; pp. 1–16.
40. Blanchet, B.; Smyth, B.; Cheval, V.; Sylvestre, M. ProVerif: Cryptographic Protocol Verifier in the Formal Model. Available online: https://bblanche.gitlabpages.inria.fr/proverif/ (accessed on 14 July 2023).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.